

Conception d'un chatbot web propulsé par un grand modèle de langage

Architecture, principes fondamentaux et implémentation

Marvin Leo

Juillet 2026

Résumé

Ce rapport présente la conception et l'implémentation d'un chatbot conversationnel avec interface web, s'appuyant sur un grand modèle de langage (*Large Language Model*, LLM) accessible via API. Au-delà de la description technique du projet, ce document a pour objectif d'expliquer les principes fondamentaux qui sous-tendent le fonctionnement d'un tel système : le fonctionnement d'un LLM, la notion de tokenisation, l'architecture client-serveur autour d'une API de génération de texte, le rôle du *prompt* système, le mécanisme de *streaming*, et la gestion de l'historique conversationnel. Le projet a été développé en Python avec le framework Streamlit pour l'interface et le SDK `google-generativeai` pour l'accès au modèle Gemini de Google.

Table des matières

1	Introduction	3
2	Principes fondamentaux	3
2.1	Qu'est-ce qu'un grand modèle de langage ?	3
2.2	Tokenisation	3
2.3	Le caractère <i>stateless</i> des LLM et la notion d'historique	4
2.4	Le <i>system prompt</i>	4
2.5	Streaming des réponses	4
2.6	Architecture client-API	4
3	Architecture et implémentation du projet	4
3.1	Stack technique	4
3.2	Organisation des fichiers	5
3.3	Boucle principale de traitement d'un message	5
4	Fonctionnalités	6
4.1	Interface conversationnelle avec streaming	6
4.2	Configuration en temps réel	6
4.3	Sauvegarde et rechargement des conversations	6
4.4	Démarrage simplifié	6
5	Choix techniques et justifications	6
5.1	Streamlit plutôt qu'un framework web classique	6
5.2	Choix du fournisseur de LLM : de Claude à Gemini	7
6	Résultat	7

7	Limites et perspectives d'évolution	8
8	Conclusion	8
A	Code source complet de l'application (app.py)	9

1 Introduction

Les grands modèles de langage (LLM) ont profondément transformé la manière dont les logiciels peuvent interagir avec le langage naturel. Il est aujourd’hui possible, avec quelques dizaines de lignes de code, de construire une application conversationnelle capable de comprendre une requête, de raisonner dessus et d’y répondre de manière cohérente – sans avoir à entraîner soi-même un modèle de *machine learning*.

Ce projet personnel avait pour objectif de concevoir un chatbot fonctionnel de bout en bout : de la réception du message de l’utilisateur jusqu’à l’affichage de la réponse générée par le modèle, en passant par la gestion de la clé d’authentification, la construction du contexte conversationnel et la persistance des échanges.

Ce rapport poursuit un double objectif :

1. présenter le résultat concret du projet : ses fonctionnalités, son architecture, les choix techniques effectués ;
2. expliquer les concepts fondamentaux qui rendent ce type de système possible, afin que le document soit compréhensible même par un lecteur peu familier avec les LLM.

2 Principes fondamentaux

Avant de détailler l’implémentation, il est utile de revenir sur les mécanismes conceptuels qui régissent le fonctionnement d’un chatbot basé sur un LLM. Cette section peut se lire comme une introduction pédagogique indépendante du projet lui-même.

2.1 Qu’est-ce qu’un grand modèle de langage ?

Un **grand modèle de langage** (LLM) est un réseau de neurones, le plus souvent basé sur l’architecture *Transformer* (Vaswani et al., 2017), entraîné sur de très grandes quantités de texte. Son objectif d’entraînement est conceptuellement simple : **prédire le mot (ou fragment de mot) suivant** dans une séquence de texte, étant donné tout ce qui précède.

En répétant cette prédiction token après token, et en réinjectant chaque token généré dans le contexte, le modèle produit des textes cohérents, capables de répondre à des questions, de résumer, de traduire ou de raisonner sur un problème – alors même qu’il n’a jamais été explicitement programmé pour effectuer ces tâches. C’est un comportement émergent de l’échelle du modèle et de la diversité des données d’entraînement.

Les modèles utilisés dans ce projet (la famille *Gemini* de Google) sont des modèles dits **propriétaires et hébergés** : ils ne tournent pas sur la machine de l’utilisateur, mais sur l’infrastructure du fournisseur, et sont accessibles via une API réseau.

2.2 Tokenisation

Un LLM ne manipule pas directement des caractères ou des mots, mais des **tokens** : des unités de sous-mots issues d’un vocabulaire fixe (typiquement quelques dizaines de milliers d’entrées). Par exemple, le mot « *chatbot* » peut être décomposé en plusieurs tokens (**chat**, **bot**), tandis qu’un mot très fréquent comme « *le* » correspond en général à un seul token.

Cette notion a deux conséquences pratiques importantes pour un développeur :

- la **facturation** des API de LLM se fait au nombre de tokens envoyés et générés, et non au nombre de caractères ou de requêtes ;

- chaque modèle a une **fenêtre de contexte** maximale (exprimée en tokens), qui limite la quantité de texte – historique de conversation compris – qu’il peut prendre en compte à un instant donné.

2.3 Le caractère *stateless* des LLM et la notion d’historique

Un point souvent contre-intuitif pour un développeur découvrant les API de LLM : **le modèle n’a pas de mémoire entre deux appels**. Chaque requête envoyée à l’API est traitée indépendamment de la précédente, sans qu’aucun état ne soit conservé côté serveur.

Pour donner l’illusion d’une conversation continue, c’est donc l’application cliente qui doit renvoyer, à chaque nouveau message, **l’intégralité de l’historique des échanges précédents**. C’est exactement le rôle que joue la variable `st.session_state.messages` dans notre application (voir section 3) : elle accumule tous les messages de la session, et cette liste complète est retransmise à l’API à chaque tour de dialogue.

Cette contrainte explique aussi pourquoi la longueur d’une conversation a un coût croissant : plus l’historique s’allonge, plus le nombre de tokens envoyés à chaque requête augmente.

2.4 Le *system prompt*

La plupart des API de LLM conversationnels distinguent le **system prompt** (ou *instruction système*) des messages de la conversation proprement dite. Le *system prompt* est un texte transmis une fois, en amont de la conversation, qui définit le comportement général attendu du modèle : son ton, son rôle, ses contraintes.

Dans notre application, ce champ est exposé directement à l’utilisateur dans l’interface (« *Tu es un assistant utile et concis.* » par défaut), ce qui permet d’expérimenter concrètement l’effet du *prompt engineering* sur le comportement du modèle sans modifier le code.

2.5 Streaming des réponses

Générer une réponse token par token prend du temps – surtout pour des réponses longues. Plutôt que d’attendre la génération complète avant de l’afficher, les API modernes proposent un mode **streaming** : le serveur renvoie les tokens au fur et à mesure de leur génération, sous forme d’un flux continu de petits fragments (*chunks*).

Côté client, il suffit d’itérer sur ce flux et de mettre à jour l’affichage à chaque fragment reçu, ce qui produit l’effet « machine à écrire » désormais familier des interfaces de chat basées sur des LLM. Ce mécanisme améliore fortement la perception de réactivité de l’application, même si le temps total de génération reste identique.

2.6 Architecture client-API

Enfin, il faut souligner que l’intelligence du système ne réside pas dans le code de l’application, mais entièrement dans le modèle hébergé par le fournisseur. L’application développée ici est un **client** : elle construit une requête HTTP (authenticée par une clé API), l’envoie au service distant, puis met en forme la réponse reçue. La figure 1 illustre ce découpage.

3 Architecture et implémentation du projet

3.1 Stack technique

Le projet repose sur trois briques principales :

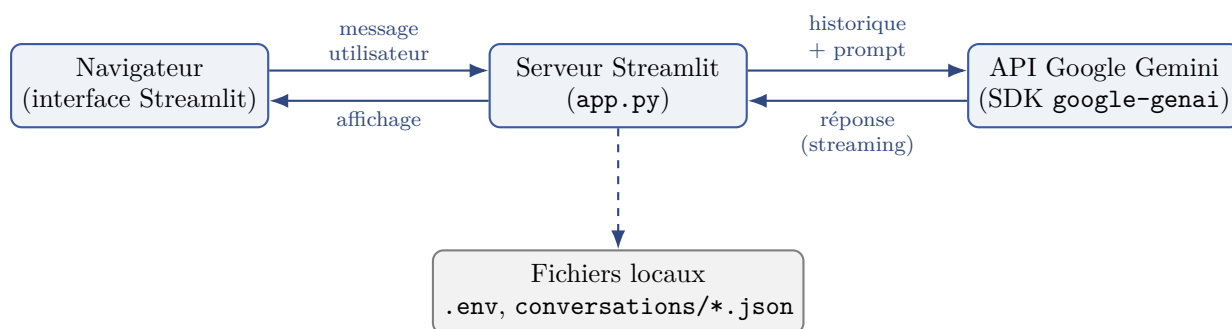


FIGURE 1 – Architecture générale du chatbot : le serveur Streamlit orchestre les échanges entre le navigateur de l'utilisateur et l'API Gemini, et gère la persistance locale (clé API, conversations sauvegardées).

- **Streamlit** : framework Python permettant de construire une interface web interactive sans écrire de HTML, CSS ou JavaScript. Streamlit ré-exécute l'intégralité du script à chaque interaction utilisateur, en conservant l'état via l'objet `st.session_state` ;
- **google-genai** : SDK Python officiel pour interroger les modèles Gemini de Google, avec support natif du streaming ;
- **python-dotenv** : chargement de variables d'environnement (notamment la clé API) depuis un fichier `.env` local, non versionné, afin de ne jamais exposer de secret dans le code source.

3.2 Organisation des fichiers

Fichier	Rôle
<code>app.py</code>	Point d'entrée de l'application (logique complète du chatbot)
<code>pyproject.toml</code>	Déclaration des dépendances du projet
<code>.env</code>	Clé API Gemini (non versionné)
<code>.env.example</code>	Modèle du fichier <code>.env</code> pour un nouvel environnement
<code>conversations/</code>	Historique des conversations sauvegardées (JSON)
<code>lancer.bat</code>	Script de démarrage en un double-clic (Windows)

3.3 Boucle principale de traitement d'un message

L'extrait suivant illustre le cœur logique de l'application : la construction du contexte conversationnel à partir de l'historique, l'appel à l'API en mode streaming, et la mise à jour progressive de l'affichage.

Listing 1 – Traitement d'un nouveau message utilisateur (extrait de `app.py`)

```

1 client = genai.Client(api_key=api_key)
2 contents = [
3     types.Content(role=m["role"], parts=[types.Part(text=m["content"])])
4     for m in st.session_state.messages
5 ]
6
7 with st.chat_message("assistant"):
8     placeholder = st.empty()
9     full_response = ""
10    stream = client.models.generate_content_stream(
11        model=MODELS[model_label],
12        contents=contents,
  
```

```
13         config=types.GenerateContentConfig(system_instruction=
14             system_prompt),
15     for chunk in stream:
16         if chunk.text:
17             full_response += chunk.text
18             placeholder.markdown(full_response + "-")
19     placeholder.markdown(full_response)
```

Deux détails techniques méritent d'être soulignés :

- la liste `contents` est reconstruite **intégralement** à chaque message, à partir de tout l'historique stocké en session – ce qui matérialise concrètement le caractère *stateless* du modèle évoqué en section 2.3;
- l'API Gemini distingue les rôles `"user"` et `"model"` (contrairement à d'autres API qui utilisent `"assistant"`); l'application effectue une conversion d'affichage pour uniformiser l'interface (voir `display_role` en annexe).

4 Fonctionnalités

4.1 Interface conversationnelle avec streaming

L'utilisateur échange des messages dans une interface de chat classique (bulles utilisateur/assistant), avec affichage progressif de la réponse au fur et à mesure de sa génération.

4.2 Configuration en temps réel

La barre latérale permet de choisir dynamiquement :

- le modèle Gemini utilisé (Flash 2.5, Pro 2.5, Flash 2.0) – un compromis vitesse/qualité/coût propre à chaque variante;
- le *system prompt*, pour expérimenter différents comportements de l'assistant;
- la clé API, préchargée automatiquement depuis le fichier `.env` si disponible.

4.3 Sauvegarde et rechargement des conversations

Une fonctionnalité de persistance a été ajoutée pour pallier la nature éphémère de l'état Streamlit (perdu à la fermeture de l'onglet) : chaque conversation peut être sauvegardée sous forme de fichier JSON horodaté dans le dossier `conversations/`, puis rechargée ultérieurement depuis un menu déroulant.

4.4 Démarrage simplifié

Un script `lancer.bat` encapsule la commande de lancement (activation de l'environnement virtuel, démarrage du serveur Streamlit), permettant un démarrage en un double-clic sans passer par un terminal.

5 Choix techniques et justifications

5.1 Streamlit plutôt qu'un framework web classique

Pour un projet personnel centré sur la logique conversationnelle plutôt que sur le design d'interface, Streamlit permet d'obtenir une interface fonctionnelle en quelques dizaines de lignes de Python, sans compétences front-end. La contrepartie est un modèle d'exécution particulier

(ré-exécution complète du script à chaque interaction) qui impose de raisonner en termes d'état persistant via `session_state`.

5.2 Choix du fournisseur de LLM : de Claude à Gemini

La première itération du projet s'appuyait sur l'API Claude d'Anthropic. Le projet a ensuite été réorienté vers l'API Gemini de Google, pour des raisons de coût : contrairement à l'API Anthropic, qui ne propose pas de palier gratuit permanent, Google met à disposition un *tier* gratuit durable (avec limites de requêtes par minute), suffisant pour un usage personnel ou une démonstration de portfolio.

Critère	API Anthropic (Claude)	API Google (Gemini)
Palier gratuit permanent	Non (crédits d'essai ponctuels)	Oui (avec limites de débit)
Carte bancaire requise	Oui	Non (tier gratuit)
SDK Python officiel	anthropic	google-genai
Support du streaming	Oui	Oui
Rôles de conversation	user / assistant	user / model

Ce changement de fournisseur n'a nécessité que la modification d'un seul fichier (`app.py`) et de la déclaration des dépendances, ce qui illustre un avantage pratique important : la logique métier (gestion de l'historique, interface, sauvegarde) est largement indépendante du fournisseur de LLM choisi.

6 Résultat

La figure 2 présente l'interface finale de l'application, avec un échange de test illustrant le fonctionnement du streaming et de la configuration en barre latérale.

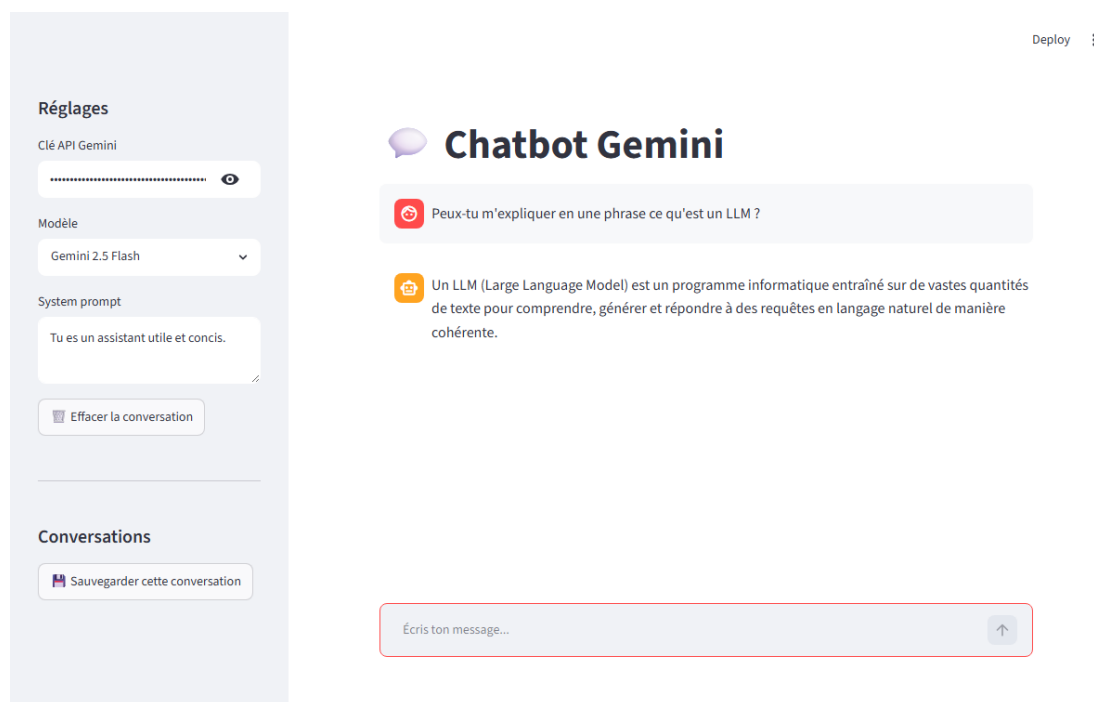


FIGURE 2 – Interface du chatbot en fonctionnement, avec un échange de test et le panneau de réglages (modèle, *system prompt*, sauvegarde des conversations).

7 Limites et perspectives d'évolution

Le projet, dans son état actuel, présente plusieurs limites assumées, qui constituent autant de pistes d'évolution naturelles :

- **Absence de mémoire long terme** : chaque session ne conserve l'historique qu'en mémoire vive ; seule une sauvegarde manuelle permet de le retrouver. Une base de données locale (SQLite) permettrait une persistance automatique.
- **Pas de gestion multi-utilisateur** : l'application est conçue pour un usage local et individuel, sans authentification.
- **Pas de recherche documentaire (RAG)** : le modèle répond uniquement à partir de ses connaissances d'entraînement et de l'historique de la conversation, sans possibilité d'interroger des documents externes.
- **Absence de contrôle de la longueur de contexte** : au-delà d'un certain volume d'échanges, l'historique complet renvoyé à l'API pourrait approcher la limite de la fenêtre de contexte du modèle ; un mécanisme de troncature ou de résumé automatique serait nécessaire pour une utilisation intensive.

Ces limites ouvrent des perspectives concrètes d'amélioration : ajout d'une recherche documentaire (*Retrieval-Augmented Generation*), gestion de plusieurs profils de conversation, ou déploiement de l'application sur un serveur accessible publiquement.

8 Conclusion

Ce projet a permis de mettre en pratique, à travers une application concrète et fonctionnelle, l'ensemble des concepts nécessaires à l'intégration d'un grand modèle de langage dans une application : gestion de l'authentification, construction du contexte conversationnel, streaming de réponses, et persistance des échanges. Le passage d'un fournisseur de LLM à un autre en cours de développement a par ailleurs illustré la portabilité de l'architecture retenue, la logique applicative restant largement indépendante du modèle sous-jacent.

A Code source complet de l'application (app.py)

Les icônes emoji de l'interface (bulle de discussion, corbeille, disquette, dossier) ont été remplacées par des étiquettes textuelles entre crochets dans cette annexe, la fonte utilisée pour la composition du document ne pouvant afficher ces glyphes.

Listing 2 – Code source intégral de app.py

```

1 import json
2 import os
3 from datetime import datetime
4 from pathlib import Path
5
6 import streamlit as st
7 from dotenv import load_dotenv
8 from google import genai
9 from google.genai import types
10
11 load_dotenv()
12
13 MODELS = {
14     "Gemini 2.5 Flash": "gemini-2.5-flash",
15     "Gemini 2.5 Pro": "gemini-2.5-pro",
16     "Gemini 2.0 Flash": "gemini-2.0-flash",
17 }
18
19 CONVERSATIONS_DIR = Path(__file__).parent / "conversations"
20 CONVERSATIONS_DIR.mkdir(exist_ok=True)
21
22 st.set_page_config(page_title="Chatbot", page_icon="chat")
23 st.title("Chatbot Gemini")
24
25 with st.sidebar:
26     st.header("Réglages")
27     api_key = st.text_input(
28         "Clé API Gemini",
29         value=os.environ.get("GEMINI_API_KEY", ""),
30         type="password",
31     )
32     model_label = st.selectbox("Modèle", list(MODELS.keys()))
33     system_prompt = st.text_area(
34         "System prompt",
35         value="Tu es un assistant utile et concis.",
36         height=100,
37     )
38     if st.button("[Effacer] Effacer la conversation"):
39         st.session_state.messages = []
40         st.rerun()
41
42     st.divider()
43     st.header("Conversations")
44
45     if st.button("[Sauvegarder] Sauvegarder cette conversation"):
46         if not st.session_state.get("messages"):
47             st.warning("Rien à sauvegarder pour l'instant.")
48         else:
49             filename = f"{datetime.now():%Y-%m-%d_%H-%M-%S}.json"
50             (CONVERSATIONS_DIR / filename).write_text(
51                 json.dumps(st.session_state.messages, ensure_ascii=False)

```

```

52         , indent=2),
53         encoding="utf-8",
54     )
55     st.success(f"Sauvegardée sous {filename}")
56
57     saved_files = sorted((f.name for f in CONVERSATIONS_DIR.glob("*.json")), reverse=True)
58     if saved_files:
59         selected_file = st.selectbox("Conversations sauvegardées",
60                                     saved_files)
61         if st.button("[Charger] Charger cette conversation"):
62             st.session_state.messages = json.loads(
63                 (CONVERSATIONS_DIR / selected_file).read_text(encoding="utf-8")
64             )
65             st.rerun()
66
67     if "messages" not in st.session_state:
68         st.session_state.messages = []
69
70     for message in st.session_state.messages:
71         display_role = "assistant" if message["role"] == "model" else
72         message["role"]
73         with st.chat_message(display_role):
74             st.markdown(message["content"])
75
76     prompt = st.chat_input("Écris ton message...")
77
78     if prompt:
79         if not api_key:
80             st.error("Merci de renseigner une clé API Gemini dans la barre latérale.")
81             st.stop()
82
83         st.session_state.messages.append({"role": "user", "content": prompt})
84         with st.chat_message("user"):
85             st.markdown(prompt)
86
87         client = genai.Client(api_key=api_key)
88         contents = [
89             types.Content(role=m["role"], parts=[types.Part(text=m["content"])]))
90         for m in st.session_state.messages
91     ]
92
93     with st.chat_message("assistant"):
94         placeholder = st.empty()
95         full_response = ""
96         stream = client.models.generate_content_stream(
97             model=MODELS[model_label],
98             contents=contents,
99             config=types.GenerateContentConfig(system_instruction=system_prompt),
100         )
101         for chunk in stream:
102             if chunk.text:
103                 full_response += chunk.text

```

```
101         placeholder.markdown(full_response + "|")
102         placeholder.markdown(full_response)
103
104     st.session_state.messages.append({"role": "model", "content":
        full_response})
```